

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 6

Roll no: 49 PRN No: 12414235 Name of Student: Saloni Tiwari

Subject Name: Deep Learning

Problem Statement:

Design and implement a Convolutional Neural Network (CNN) for image classification using the Tomato or Soybean disease dataset.

Algorithm:

- Import libraries for data handling, image processing, CNN modeling, hyperparameter tuning, visualization, and evaluation.
- Load the Rice Leaf Disease dataset and assign images to their corresponding disease classes.
- Explore the dataset, check class distribution, sample images, and missing values.
- Split the dataset into training, validation, and testing sets using a 70:15:15 ratio.
- Resize and normalize images and apply data augmentation to improve generalization.
- Define a CNN with convolution, pooling, dense, dropout, and softmax layers.
- Use Keras Tuner Random Search to optimize filters, CNN layers, dense units, dropout, and learning rate.
- Select and train the CNN configuration with the best validation accuracy.
- Evaluate the optimized model on the test dataset and calculate accuracy and loss.
- Plot training/validation accuracy and loss to analyze model performance.
- Generate the classification report and confusion matrix to evaluate disease-wise performance.
- Compare actual and predicted labels and save the optimized CNN model.

Code:

```
# 1. Import libraries
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
import keras_tuner as kt
from PIL import Image
```

```

from tensorflow.keras import Sequential
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# 2. Loading the dataset
dataset_dir = "rice_leaf_diseases"
print("Dataset exists:", os.path.exists(dataset_dir))
print("Classes:", os.listdir(dataset_dir))

# 3. Creating the dataset dataframe
data = []
for class_name in os.listdir(dataset_dir):
    class_path = os.path.join(dataset_dir, class_name)
    if os.path.isdir(class_path):
        for file_name in os.listdir(class_path):
            if file_name.lower().endswith((".jpg", ".jpeg", ".png")):
                data.append({
                    "filepath": os.path.join(class_path, file_name),
                    "label": class_name
                })
df = pd.DataFrame(data)
print(df.head())
print("Total images:", len(df))

# 4. Dataset information
print(df.info())
print(df.describe())
print(df["label"].value_counts())

# 5. Checking missing values
print(df.isnull().sum())

# 6. Visualizing class distribution
plt.figure(figsize=(8, 5))
sns.countplot(
    data=df,
    x="label"
)
plt.xlabel("Disease")
plt.ylabel("Number of Images")
plt.title("Rice Leaf Disease Class Distribution")
plt.xticks(rotation=20)
plt.show()

# 7. Displaying sample images
classes = df["label"].unique()
plt.figure(figsize=(12, 5))
for i, class_name in enumerate(classes):
    sample = df[df["label"] == class_name].iloc[0]
    image = Image.open(sample["filepath"])
    plt.subplot(1, len(classes), i + 1)
    plt.imshow(image)
    plt.title(class_name)
    plt.axis("off")
plt.tight_layout()
plt.show()

# 8. Training, validation and testing splitting
train_df, temp_df = train_test_split(

```

```

    df,
    test_size=0.30,
    stratify=df["label"],
    random_state=42
)
val_df, test_df = train_test_split(
    temp_df,
    test_size=0.50,
    stratify=temp_df["label"],
    random_state=42
)
print("Training images:", len(train_df))
print("Validation images:", len(val_df))
print("Testing images:", len(test_df))

# 9. Image preprocessing and augmentation
IMG_SIZE = (128, 128)
BATCH_SIZE = 16
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=25,
    width_shift_range=0.15,
    height_shift_range=0.15,
    zoom_range=0.20,
    shear_range=0.15,
    horizontal_flip=True,
    fill_mode="nearest"
)
test_datagen = ImageDataGenerator(
    rescale=1./255
)

# 10. Creating image generators
train_generator = train_datagen.flow_from_dataframe(
    train_df,
    x_col="filepath",
    y_col="label",
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode="categorical",
    shuffle=True,
    seed=42
)
val_generator = test_datagen.flow_from_dataframe(
    val_df,
    x_col="filepath",
    y_col="label",
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode="categorical",
    shuffle=False
)
test_generator = test_datagen.flow_from_dataframe(
    test_df,
    x_col="filepath",
    y_col="label",
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode="categorical",
    shuffle=False
)

```

```
# 11. Defining classes
class_names = sorted(train_df["label"].unique())
num_classes = len(class_names)
print("Classes:", class_names)
print("Number of classes:", num_classes)
```

```
# 12. Creating the AutoTune CNN
```

```
def build_model(hp):
    model = Sequential()
    model.add(
        Input(shape=(128, 128, 3))
    )
    num_conv_blocks = hp.Int(
        "num_conv_blocks",
        min_value=2,
        max_value=3,
        step=1
    )
    for i in range(num_conv_blocks):

        filters = hp.Choice(
            f"filters_{i}",
            values=[32, 64, 128]
        )
        model.add(
            Conv2D(
                filters=filters,
                kernel_size=3,
                activation="relu",
                padding="same"
            )
        )
        model.add(
            MaxPooling2D(pool_size=2)
        )
    model.add(Flatten())
    dense_units = hp.Choice(
        "dense_units",
        values=[64, 128, 256]
    )
    model.add(
        Dense(
            dense_units,
            activation="relu"
        )
    )
    dropout = hp.Float(
        "dropout",
        min_value=0.2,
        max_value=0.5,
        step=0.1
    )
    model.add(
        Dropout(dropout)
    )
    model.add(
        Dense(
            num_classes,
            activation="softmax"
        )
    )
    learning_rate = hp.Choice(
```

```

        "learning_rate",
        values=[0.001, 0.0005, 0.0001]
    )
    model.compile(
        optimizer=tf.keras.optimizers.Adam(
            learning_rate=learning_rate
        ),
        loss="categorical_crossentropy",
        metrics=["accuracy"]
    )
    return model

# 13. Creating the AutoTune tuner
tuner = kt.RandomSearch(
    build_model,
    objective="val_accuracy",
    max_trials=8,
    directory="autotune_results",
    project_name="rice_leaf_cnn",
    overwrite=True
)

# 14. Running AutoTune
tuner.search(
    train_generator,
    validation_data=val_generator,
    epochs=10
)

# 15. Getting the best hyperparameters
best_hp = tuner.get_best_hyperparameters(
    num_trials=1
)[0]
print("Best Hyperparameters:")
for parameter, value in best_hp.values.items():
    print(parameter, ":", value)

# 16. Getting the best CNN model
best_model = tuner.get_best_models(
    num_models=1
)[0]
best_model.summary()

# 17. Training the best model
early_stopping = EarlyStopping(
    monitor="val_loss",
    patience=7,
    restore_best_weights=True
)
history = best_model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=30,
    callbacks=[early_stopping]
)

# 18. Evaluating accuracy and plotting results
test_loss, test_accuracy = best_model.evaluate(
    test_generator
)
print("Test Loss:", test_loss)

```

```

print("Test Accuracy:", test_accuracy)
plt.figure(figsize=(8, 5))
plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)
plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training vs Validation Accuracy")
plt.legend()
plt.show()
plt.figure(figsize=(8, 5))
plt.plot(
    history.history["loss"],
    label="Training Loss"
)
plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training vs Validation Loss")
plt.legend()
plt.show()

```

19. Predictions, classification report and confusion matrix

```

test_generator.reset()
y_prob = best_model.predict(
    test_generator
)
y_pred = np.argmax(
    y_prob,
    axis=1
)
y_test = test_generator.classes
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=class_names,
        zero_division=0
    )
)
cm = confusion_matrix(
    y_test,
    y_pred
)
plt.figure(figsize=(7, 6))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=class_names,
    yticklabels=class_names
)

```

```
)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
results = pd.DataFrame({
    "Actual": [class_names[i] for i in y_test],
    "Predicted": [class_names[i] for i in y_pred]
})
print("\nActual vs Predicted:")
print(results.head(15))
```

Output:

1]

```
#import libraries
import os
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import tensorflow as tf
import keras_tuner as kt

from PIL import Image

from tensorflow.keras import Sequential
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import EarlyStopping

from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
```

✓ 0.0s

2]

```
#Load the dataset
dataset_dir = "DeepLearning/rice_leaf_diseases"

print("Dataset exists:", os.path.exists(dataset_dir))
print("Folders:", os.listdir(dataset_dir))
```

✓ 0.0s

```
Dataset exists: True
Folders: ['Bacterial leaf blight', 'Brown spot', 'Leaf smut']
```

3]

```
#create a dataframe to hold the image paths and labels
data = []

for class_name in os.listdir(dataset_dir):
    class_path = os.path.join(dataset_dir, class_name)

    if os.path.isdir(class_path):
        for file_name in os.listdir(class_path):
            if file_name.lower().endswith((".jpg", ".jpeg", ".png")):
                data.append({
                    "filepath": os.path.join(class_path, file_name),
                    "label": class_name
                })

df = pd.DataFrame(data)

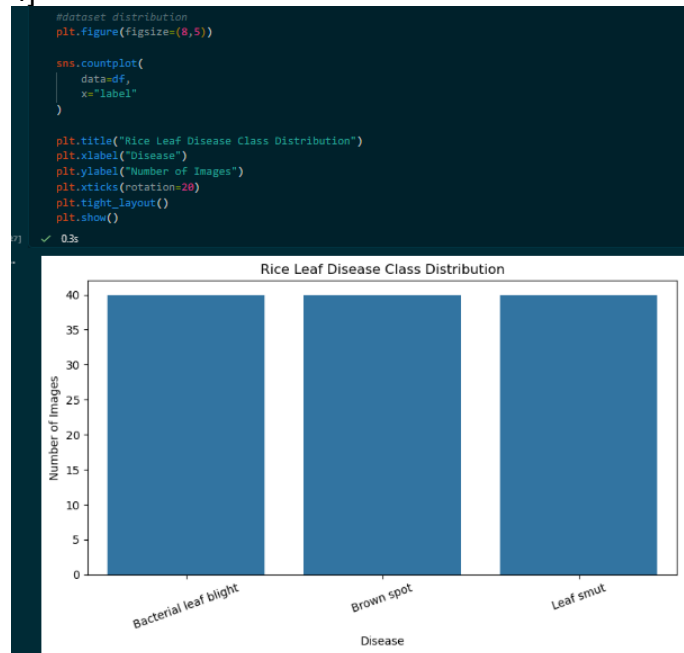
print("Total images:", len(df))
print("Classes:")
print(df["label"].value_counts())
```

✓ 0.0s

```
Total images: 120
```

```
Classes:
label
Bacterial leaf blight    40
Brown spot               40
Leaf smut                40
Name: count, dtype: int64
```

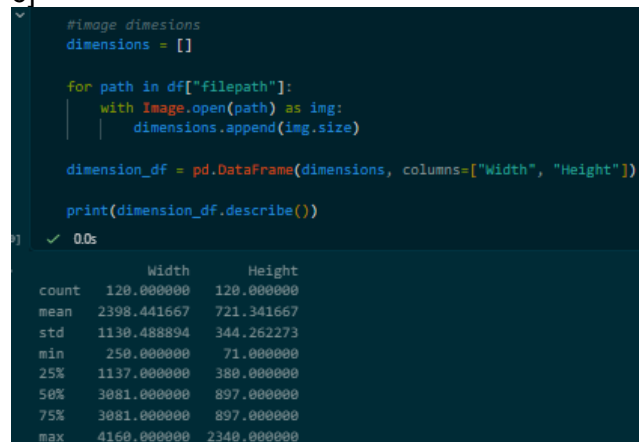
4]



5]



6]



7]

```

Training: 84
Validation: 18
Testing: 18

Found 84 validated image filenames belonging to 3 classes.
Found 18 validated image filenames belonging to 3 classes.
Found 18 validated image filenames belonging to 3 classes.

Classes: ['Bacterial leaf blight', 'Brown spot', 'Leaf smut']
Number of classes: 3

```

8] Building auto-tune CNN, creating a tuner and running the auto-tune

```

Trial 8 Complete [00h 01m 19s]
val_accuracy: 0.4444444477558136

Best val_accuracy So Far: 0.66666666865348816
Total elapsed time: 00h 05m 57s

```

9]

```

# best hyperparameters and best model
best_hp = tuner.get_best_hyperparameters(num_trials=1)[0]

print("Best Hyperparameters:")
for parameter, value in best_hp.values.items():
    print(f"{parameter}: {value}")

best_model = tuner.get_best_models(num_models=1)[0]

best_model.summary()
✓ 1.1s

Best Hyperparameters:
num_conv_blocks: 3
filters_0: 32
filters_1: 64
dense_units: 128
dropout: 0.30000000000000004
learning_rate: 0.0005
filters_2: 32
c:\Users\DELL\anaconda1\lib\site-packages\keras\src\saving\saving_lib.py:869: UserWarning: Skipping variable loading for optimizer 'adam', because it has 2 variables whereas the saved optimizer has 22 variables.
  saveable.load_own_variables(store)

Model: "sequential"

```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 320, 320, 32)	896
max_pooling2d (MaxPooling2D)	(None, 64, 64, 32)	0
conv2d_1 (Conv2D)	(None, 64, 64, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 32, 32, 64)	0
conv2d_2 (Conv2D)	(None, 32, 32, 32)	18,464
max_pooling2d_2 (MaxPooling2D)	(None, 16, 16, 32)	0
flatten (Flatten)	(None, 8192)	0
dense (Dense)	(None, 128)	1,048,704
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 3)	387

```

Total params: 1,049,947 (4.15 MB)

Trainable params: 1,049,947 (4.15 MB)

Non-trainable params: 0 (0.00 B)

```

10] Training the best model

```
# training the best model
early_stopping = EarlyStopping(
    monitor="val_loss",
    patience=7,
    restore_best_weights=True
)

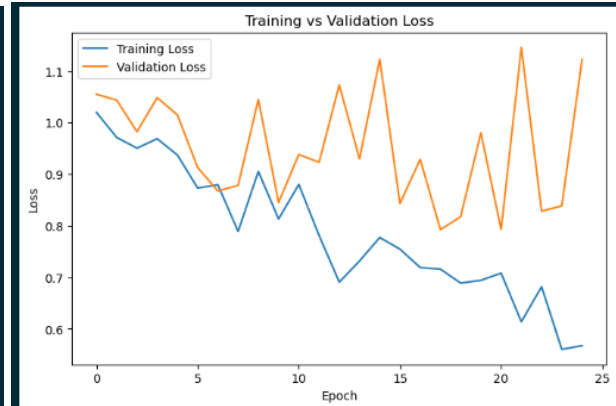
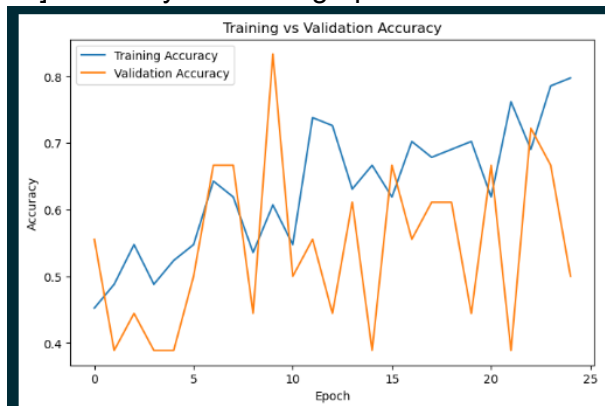
history = best_model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=30,
    callbacks=[early_stopping]
)
```

✓ 1m 35.0s

Epoch 1/30
6/6 ————— 7s 795ms/step - accuracy: 0.4524 - loss: 1.0197 - val_accuracy: 0.5556 - val_loss: 1.0551
Epoch 2/30
6/6 ————— 4s 700ms/step - accuracy: 0.4881 - loss: 0.9712 - val_accuracy: 0.3889 - val_loss: 1.0436
Epoch 3/30
6/6 ————— 4s 608ms/step - accuracy: 0.5476 - loss: 0.9505 - val_accuracy: 0.4444 - val_loss: 0.9824
Epoch 4/30
6/6 ————— 4s 576ms/step - accuracy: 0.4881 - loss: 0.9687 - val_accuracy: 0.3889 - val_loss: 1.0484
Epoch 5/30
6/6 ————— 4s 598ms/step - accuracy: 0.5238 - loss: 0.9371 - val_accuracy: 0.3889 - val_loss: 1.0149
Epoch 6/30
6/6 ————— 4s 584ms/step - accuracy: 0.5476 - loss: 0.8732 - val_accuracy: 0.5000 - val_loss: 0.9130
Epoch 7/30
6/6 ————— 4s 688ms/step - accuracy: 0.6429 - loss: 0.8795 - val_accuracy: 0.6667 - val_loss: 0.8672
Epoch 8/30
6/6 ————— 4s 585ms/step - accuracy: 0.6190 - loss: 0.7890 - val_accuracy: 0.6667 - val_loss: 0.8784
Epoch 9/30
6/6 ————— 4s 658ms/step - accuracy: 0.5357 - loss: 0.9052 - val_accuracy: 0.4444 - val_loss: 1.0449
Epoch 10/30
6/6 ————— 3s 545ms/step - accuracy: 0.6071 - loss: 0.8131 - val_accuracy: 0.8333 - val_loss: 0.8451
Epoch 11/30
6/6 ————— 4s 607ms/step - accuracy: 0.5476 - loss: 0.8801 - val_accuracy: 0.5000 - val_loss: 0.9382
Epoch 12/30
6/6 ————— 4s 569ms/step - accuracy: 0.7381 - loss: 0.7813 - val_accuracy: 0.5556 - val_loss: 0.9230
Epoch 13/30
...
Epoch 24/30
6/6 ————— 4s 601ms/step - accuracy: 0.7857 - loss: 0.5601 - val_accuracy: 0.6667 - val_loss: 0.8385
Epoch 25/30
6/6 ————— 4s 634ms/step - accuracy: 0.7976 - loss: 0.5673 - val_accuracy: 0.5000 - val_loss: 1.1227

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings.

11] Accuracy and Loss graphs



12] Testing Evaluation + Classification Report

```
2/2 ————— 1s 40ms/step - accuracy: 0.7222 - loss: 0.5846  
Test Loss: 0.5846129655838013  
Test Accuracy: 0.722222089767456  
2/2 ————— 1s 112ms/step
```

Classification Report:

	precision	recall	f1-score	support
Bacterial leaf blight	0.60	1.00	0.75	6
Brown spot	0.75	0.50	0.60	6
Leaf smut	1.00	0.67	0.80	6
accuracy			0.72	18
macro avg	0.78	0.72	0.72	18
weighted avg	0.78	0.72	0.72	18

13] Confusion Matrix

